



PHP POO 2025

Silvio Priem Mendes, PhD

PHP OOP

- **What does OOP aim to achieve?**
 - Allow compartmentalized refactoring of code.
 - Promote code re-use.
 - Promote extensibility, flexibility and adaptability.
 - Better for team development.
 - Many patterns are designed for OOP.
 - Some patterns lead to much more efficient code.
- **Do you need to use OOP to achieve these goals?**
 - Of course not.
 - Although It's designed to make things easier.

PHP OOP

Concepts

- **Class:** This is a programmer-defined datatype, which includes local functions as well as local data. You can think of a class as a template for making many instances of the same kind (or class) of object.
- **Object:** An individual instance of the data structure defined by a class. You define a class once and then make many objects that belong to it. Objects are also known as instance.
- **Member Variable:** These are the variables defined inside a class. This data will be invisible to the outside of the class and can be accessed via member functions. These variables are called attribute of the object once an object is created.
- **Method:** These are the function defined inside a class and are used to access object data.

PHP OOP

- Class Example:

```
class SuperPerson
{
    public $name; // Accessible to anyone
    public $powerLevel;

    // Constructor
    public function __construct($name, $powerLevel = 0) {
        $this->name = $name;
        $this->powerLevel = $powerLevel;
    }

    // Destructor called when object goes out of scope
    function __destruct() {
        echo "Bye-bye";
    }

    // Convert object to string representation
    public function __toString() {
        return "Name = $this->name, PowerLevel = $this->powerLevel";
    }
}
```

PHP OOP

- Subclass Example:

```
class SuperHero extends SuperPerson
{
    private $savedVictims = 0; // Accessible only within the class

    public function save($victim) {
        echo $this->name .'is saving $victim.';
        $savedVictims++;
    }

    public function getNumberOfSavedVictims() {
        return $savedVictims;
    }
}
```

PHP OOP

- Usage Example:

```
$hero = new SuperHero("Spam-Man", 3); //SuperHero instance  
  
echo "$hero"; // Prints Name = Spam-Man, PowerLevel = 3  
  
$hero->save("Laura Jones");
```

PHP OOP

- **Visibility**
 - **Public** (everyone accesses)
 - **Protected** (owner and subclass)
 - **Private** (owner)
- **Constructor** (good for initializing values)
- **Destructor** (release any resource that has been used)



BIBLIOGRAPHY